Andrew Kozlevcar

11:00 to 11:50

In [7]:
```
from pathlib import Path
from io import StringIO
import pandas as pd
import numpy as np
import plotly.express as px
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from IPython.display import display

# The supplied notebook includes an embedded copy of the data.
# For a blank notebook, upload slump_test.csv or use the UCI URL.
if "SLUMP_CSV" in globals():
    raw = pd.read_csv(StringIO(SLUMP_CSV))
elif Path("slump_test.csv").is_file():
    raw = pd.read_csv("slump_test.csv")
else:
    url = ("https://archive.ics.uci.edu/ml/"
           "machine-learning-databases/concrete/slump/"
           "slump_test.data")
    raw = pd.read_csv(url)

reference = raw.rename(columns={
    "No": "mix_id", "Cement": "cement", "Slag": "slag",
    "Fly ash": "fly_ash", "Water": "water",
    "SP": "superplasticizer", "Coarse Aggr.": "coarse_agg",
    "Fine Aggr.": "fine_agg", "SLUMP(cm)": "slump",
    "FLOW(cm)": "flow",
    "Compressive Strength (28-day)(Mpa)": "strength"
}).copy()

print("Original data shape:", reference.shape)
display(reference.head())
display(reference.isna().sum().rename("Original missing count"))
assert reference.shape == (103, 11)
assert not reference.isna().any().any()

# A teaching grouping, NOT an engineering standard.
reference["coarse_group"] = np.where(
    reference["coarse_agg"] >= 900,
    "At least 900", "Below 900"
)

fig = px.scatter(
    reference, x="coarse_agg", y="water",
    color="coarse_group", symbol="coarse_group",
    hover_data=["mix_id"],
    labels={"coarse_agg": "Coarse aggregate (kg/m^3)",
```

```
            "water": "Water (kg/m^3)",
            "coarse_group": "Coarse aggregate group"},
    title="Before hiding values: recorded concrete ingredients"
)
fig.show()
```

```
Original data shape: (103, 11)
```

	mix_id	cement	slag	fly_ash	water	superplasticizer	coarse_agg	fine_agg	slum
0	1	273.0	82.0	105.0	210.0	9.0	904.0	680.0	23.
1	2	163.0	149.0	191.0	180.0	12.0	843.0	746.0	0.
2	3	162.0	148.0	191.0	179.0	16.0	840.0	743.0	1.
3	4	162.0	148.0	190.0	179.0	19.0	838.0	741.0	3.
4	5	154.0	112.0	144.0	220.0	10.0	923.0	658.0	20.

```
mix_id              0
cement              0
slag                0
fly_ash             0
water               0
superplasticizer    0
coarse_agg          0
fine_agg            0
slump               0
flow                0
strength            0
Name: Original missing count, dtype: int64
```

Question 1

In [8]:
```
# Artificial rule (ii): probability depends on observed coarse_agg.
# Keep this seed and these probabilities unchanged.
rng = np.random.default_rng(123)
p_missing = np.where(reference["coarse_agg"] >= 900,
                     0.60, 0.15)
mask = rng.random(len(reference)) < p_missing

missing = reference.copy()
missing.loc[mask, "water"] = np.nan
missing["water_missing"] = mask
observed = missing.dropna(subset=["water"]).copy()

print("Artificially hidden water entries:", int(mask.sum()))
print("Complete cases retained:", len(observed))

rates = missing.groupby("coarse_group")["water_missing"].agg(
    Records="size", Hidden="sum", Rate="mean"
)
rates["Missing percent"] = 100 * rates["Rate"]
display(rates[["Records", "Hidden", "Missing percent"]].round(2))
```

```
shares = pd.DataFrame({
    "Reference percent": 100 * reference["coarse_group"].value_counts(
        normalize=True),
    "Complete-case percent": 100 * observed["coarse_group"].value_counts(
        normalize=True)
})
display(shares.round(2))

fig = px.bar(
    rates.reset_index(), x="coarse_group", y="Missing percent",
    labels={"coarse_group": "Coarse aggregate group"},
    title="Realized missingness under the artificial rule"
)
fig.show()
```

```
Artificially hidden water entries: 35
Complete cases retained: 68
```

coarse_group	Records	Hidden	Missing percent
At least 900	42	27	64.29
Below 900	61	8	13.11

coarse_group	Reference percent	Complete-case percent
Below 900	59.22	77.94
At least 900	40.78	22.06

Part A

There are 103 source records, and the original missing-value count is zero. Generally speaking, deliberately hiding known values has two advantages. First, unlike the real world, we can control the missing mechanism (for instance, choosing one of missing strategies described above). Additionally, since we have the original data, we can see how missingness alters the shape of the remaining data relative to the original data, and we can see how imputation strategies restore the data given the various types of missingness we impose.

Part B

Rule 1 is missing completely at random (MCAR); this is because the missingness of water has a 30% of being hidden, regardless of its value OR observed values. That is, missingness does not depend on observed or unobserved data.

Rule 2 is missing at random (MAR); this is because the probability of missingness for a given entry of water is only dependent on the observed coarse aggregate. That is, missingness only depends on observed data.

Rule 3 is missing not at random (MNAR). This is because the probability of missingness for a given entry of water is dependent on the missing water value itself; higher water values are more likely to be hidden. That is, missingness depends on the unobserved data.

Part C

The missing percent for coarse aggregate below 900 kg/m^3 is about 13.11%, whereas the missing percent for atleast 900 kg/m^3 coarse aggregate is about 64.29%; these rates are clearly very different, and it would be implausible to produce such a discrepency by chance. This suggests that missingness is associated with coarse aggregate.

Different rates across groups in a real dataset would not prove MAR rather than MNAR. MAR is really a two-sided statement; missingness must depend on observed values and NOT depend on the unobserved values. On the other hand, MNAR requires that the missigness depend on unobserved values. The point is that both make claims about depenedency with respect to unobserved data. Of course, we do not have this unobserved data, so we cannot test the stated conditions.

Question 2

In [9]:
```
observed_mean = observed["water"].mean()
mean_data = missing.copy()
mean_data["water"] = mean_data["water"].fillna(observed_mean)
print("One value used for every gap:", round(observed_mean, 3))

frames = {
    "Reference (benchmark)": reference,
    "A: Complete cases": observed,
    "B: Mean imputation": mean_data
}

rows = []
for name, frame in frames.items():
    rows.append({
        "Dataset": name, "Rows": len(frame),
        "Mean water": frame["water"].mean(),
        "SD water": frame["water"].std()
    })
water_summary = pd.DataFrame(rows)
display(water_summary.round(3))
```

```
variance_ratio = mean_data["water"].var() / observed["water"].var()
identity_ratio = (len(observed) - 1) / (len(reference) - 1)
print("Completed / observed variance:", round(variance_ratio, 4))
print("(n_observed - 1) / (n - 1):", round(identity_ratio, 4))

mean_data["Entry"] = np.where(mask, "Imputed", "Observed")
fig = px.scatter(
    mean_data, x="coarse_agg", y="water", color="Entry",
    symbol="Entry", hover_data=["mix_id"],
    labels={"coarse_agg": "Coarse aggregate (kg/m^3)",
            "water": "Water (kg/m^3)"},
    title="Mean imputation: examine the imputed entries"
)
fig.show()
```

```
One value used for every gap: 201.89
```

	Dataset	Rows	Mean water	SD water
0	Reference (benchmark)	103	197.168	20.208
1	A: Complete cases	68	201.890	18.860
2	B: Mean imputation	103	201.890	15.285

```
Completed / observed variance: 0.6569
(n_observed - 1) / (n - 1): 0.6569
```

Part A

68 cases are completely retained. Before deletion, the below 900 coarse group comprised 59.22% of the cases, whereas the at least 900 kg/m^3 coarse group comprised 40.78%. After deletion, the below 900 kg/m^3 coarse group comprised 77.94% of the cases, whereas the at least 900 kg/m^3 coarse group comprised 22.06%. That is, the at least $900 kg/m^3$ coarse group saw a roughly 19 percentage point decrease in representation in the complete case. This makes sense. Indeed, the higher coarse group had a higher conditional probability of deletion (4 times the probability) than the lower coarse group. Thus, we should expect a decrease in percentage representation in the higher coarse group.

Part B

The mean of the complete-case and mean-imputed remain identical at $201.890 kg/m^3$, which makes sense; repeatedly adding the average to a sample does not change the average. However, the water standard deivation went from $18.860 kg/m^3$ in the complete case to $15.285 kg/m^3$ in the mean imputed case. Since all of the imputed values were the mean, they contribute 0 to the numerator of the sample variance calculation, yet they contribute an additional 35 in the denominator; the result is a smaller variance and a smaller corresponding standard deviation. Visually, we see the imputed means form a horizontal line, "redistributing" more of the sample mass to the

mean of the data. As a result, more of the plot congregates in the center, and the plot appears less varied than the complete case.

Mean imputing does not restore the original distribution for several reasons. First, mean-imputing uses the remaining data to rebuild the original data. In this case, since our remaining data was not representative of the original data (because removals systematically favored the high coarse group), our observed mean (201.890 kg/m^3) was biased upward compared to the reference (197.168 kg/m^3) (because we disproportionately removed the high coarse group, who tend to have lower water values). When we replace missing values with the sample mean, all we can do is reproduce this bias. Likewise, mean-imputing assumes that the replaced points sit exactly at the observed mean, which is unrealistic in virtually all settings. In this case, we see that an unconditional mean-imputation is inappropriate because the scatter plot demonstrates a negative relationship between water and coarse aggregate. Thus, we understate the spread of the distribution by providing all points the mean, regardless of their coarse aggregate status. Even if we did account for this negative relationship, points would also possess slight perturbations, which contribute to variation as well. Since our imputations are deterministic and fixed, we do not simulate this type of variation, either. Mathematically, these added points contribute 0 squared deviation, which is the minimum they can contribute to sample variance. As a testament to this, the reference standard deviation is 20.208 kg/m^3 compared to the imputation standard deviation of 15.285 kg/m^3.

Part C

While the median measure is generally less sensitive to extreme values, this robust quality is, in some sense, not relevant to handling systematic missingness. If an imputation is fixed and based on a biased sample, it cannot magically recover ommited information, and its static/fixed nature does not allow it to model variable correlation (the negative relationship between coarse aggregate and water, for instance).

The points from 2b generally apply to a median imputation. Indeed, since our remaining data was not representative of the original data (because removals systematically favored the high coarse group), our observed median is likely still biased upward compared to the reference median (because we disproportionately removed the high coarse group, who tend to have lower water values). When we replace missing values with the sample median, all we can do is reproduce this bias; unlike the mean case, though, the imputed median values may slightly change the mean, though this change is driven by our biased information. As described in greater detail in the previous response, a fixed imputation does not respect the negative relationship between coarse aggregate and water, understating the spread of the imputed values as a result. Even if we did account for this negative relationship, points would also possess slight perturbations, which contribute to variation as well. Since our median imputations are deterministic, we

do not simulate this type of variation, either. Mathematically, it can be shown that median imputation will possibly lead to some squared deviation contribution (the median is not neccesarily equal to the mean, meaning new points can have non-zero squared deviation, and the total squared deviations of prior points will also grow with an offset mean), but we still underestimate the variance of the distirbution.

Question 3

In [10]:
```
# Only observed ingredients enter the imputation model.
# Do not use strength, slump, flow, mix_id, or hidden water values.
imputer_features = [
    "cement", "slag", "fly_ash", "superplasticizer",
    "coarse_agg", "fine_agg"
]
imputer = LinearRegression()
imputer.fit(observed[imputer_features], observed["water"])

reg_data = missing.copy()
reg_data.loc[mask, "water"] = imputer.predict(
    missing.loc[mask, imputer_features]
)

# Reveal true water values ONLY to evaluate the hidden positions.
recovery_rows = []
for name, frame in {
    "B: Mean imputation": mean_data,
    "C: Regression imputation": reg_data
}.items():
    rmse = np.sqrt(mean_squared_error(
        reference.loc[mask, "water"], frame.loc[mask, "water"]
    ))
    recovery_rows.append({
        "Method": name, "Hidden entries evaluated": int(mask.sum()),
        "Recovery RMSE (kg/m^3)": rmse
    })
recovery = pd.DataFrame(recovery_rows)
display(recovery.round(3))

frames["C: Regression imputation"] = reg_data
rows = []
for name, frame in frames.items():
    rows.append({
        "Dataset": name, "Rows": len(frame),
        "Mean water": frame["water"].mean(),
        "SD water": frame["water"].std()
    })
water_summary = pd.DataFrame(rows)
display(water_summary.round(3))

reg_data["Entry"] = np.where(mask, "Imputed", "Observed")
fig = px.scatter(
    reg_data, x="coarse_agg", y="water", color="Entry",
```

```
    symbol="Entry", hover_data=["mix_id"],
    labels={"coarse_agg": "Coarse aggregate (kg/m^3)",
            "water": "Water (kg/m^3)"},
    title="Regression imputation: compare with the mean-filled plot"
)
fig.show()

# Plot each method separately, with the same axes.
for name, frame in {
    "Mean imputation": mean_data,
    "Regression imputation": reg_data
}.items():
    check = pd.DataFrame({
        "Actual water": reference.loc[mask, "water"],
        "Imputed water": frame.loc[mask, "water"]
    })
    fig = px.scatter(
        check, x="Actual water", y="Imputed water",
        range_x=[150, 245], range_y=[150, 245],
        labels={"Actual water": "Actual water (kg/m^3)",
                "Imputed water": "Imputed water (kg/m^3)"},
        title=name + ": recovery of hidden entries"
    )
    fig.add_shape(type="line", x0=150, y0=150, x1=245, y1=245,
                  line=dict(dash="dash"))
    fig.show()
```

	Method	Hidden entries evaluated	Recovery RMSE (kg/m^3)
0	B: Mean imputation	35	23.976
1	C: Regression imputation	35	4.553

	Dataset	Rows	Mean water	SD water
0	Reference (benchmark)	103	197.168	20.208
1	A: Complete cases	68	201.890	18.860
2	B: Mean imputation	103	201.890	15.285
3	C: Regression imputation	103	196.921	20.548

Part A

The response of the imputation model is water (kg/m^3), whereas the predictors are cement, slag, fly_ash, superplasticizer, coarse_agg, fine_agg. While strength is available in a retrospective analysis, it will not be available at prediction time. That is, the goal is to produce a model that predicts strength before we observe it. Therefore, if our model requires an input of observed strength for imputation, our model becomes unusable in a non-retrospective setting.

Besides the prediction-time issue, the deterministic imputation setting also poses problems with using the outcome as an imputation predictor, namely an overstatement of water's effect. If we performed an imputation with strength, then our imputed water values would roughly be a function of the strength, which will in turn make water's effect stronger than it actually is; even if the relationship did hold, it would never look this "perfect". However, omitting strength all together runs the risk of understating the effect of water in the final model by not capturing any of the direct relationship between strength and water implied by the observed data; a compromise would therefore involve imputation using the outcome but appropriately perturbing the imputation to avoid overstating the effect.

Part B

According to the table, the recovery RMSE of mean imputation is 23.976 kg/m^3, whereas the recovery RMSE of regression imputation is 4.553 kg/m^3. Based on both the plot and the recovery RMSE, I would choose the regression imputation. The recovery RMSE is self-explanatory; smaller RMSE means better recovery in this experiment, and so regression imputation does a better job at recovering the missing data. The recovery plot conveys this message but in a more nuanced way. Moreover, observe that the regression imputed values all straddle the 45 degree line quite tightly, suggesting that imputed water is generally quite close to the actual water. For the most part, the imputation appears uniform; in each region of actual water, the imputation overestimates and underestimates values. The possible exception is the region from 200 to 220 kg/m^3, where impuited water tend to overestimate slightly. This contrasts with the mean imputation, where there are large overestimates in the low actual water region and large underestimates in the high actual water region.

Obviously, there are several limitations. The first is that we performed a single missingness simulation, which means our estimator will be relatively variable. Indeed, a second simulation may produce very different results. Next, this imputation process fundamentally relies on the unverifiable assumption of MAR, which leaves us to wonder whether our imputations may be biased (systematically overestimates or underestimates); we have no way of demonstrating our uncertainty and, instead, the points are treated as fact. Additionally, we have not accounted for individual perturbations, meaning our imputation strategy is still understating variation and not recovering the original distribution as well as it could. Finally, good imputation need not imply that our model is good at predicting the desired outcome of strength.

Part C

For the observed rows, there is no imputation being done, meaning that they would contribute a zero to the sum but artifically reduce the RMSE value by introducing a larger

denominator. This would complicate our metric because the outputted value now depends on the proportion of missing data, which a selected imputation strategy should not be held accountable for; this is inherent to the problem. Obviously, deletion has no recovery RMSE because deletion involves no imputation; there are no comparisons to be done. Finally, the recovery RMSE metric relies on knowledge of the true water values. In a non-toy setting, these are the values that are missing, and so there would be no way to perform this computation.

Question 4

In [11]:
```
# Same later regression in every dataset:
# strength = intercept + b1 * water + b2 * cement + error.
# All metrics below are IN-SAMPLE; there is no strength test set.
strength_features = ["water", "cement"]
model_rows = []
for name, frame in frames.items():
    model = LinearRegression()
    model.fit(frame[strength_features], frame["strength"])
    fitted = model.predict(frame[strength_features])
    model_rows.append({
        "Dataset": name, "Rows": len(frame),
        "Water coefficient": model.coef_[0],
        "Cement coefficient": model.coef_[1],
        "In-sample R2": model.score(
            frame[strength_features], frame["strength"]),
        "In-sample RMSE (MPa)": np.sqrt(mean_squared_error(
            frame["strength"], fitted))
    })
strength_summary = pd.DataFrame(model_rows)
display(strength_summary.round(4))
```

	Dataset	Rows	Water coefficient	Cement coefficient	In-sample R2	In-sample RMSE (MPa)
0	Reference (benchmark)	103	-0.1439	0.0524	0.3295	6.3869
1	A: Complete cases	68	-0.0825	0.0452	0.2557	6.2173
2	B: Mean imputation	103	-0.0831	0.0462	0.2245	6.8688
3	C: Regression imputation	103	-0.1425	0.0531	0.3303	6.3830

Part A

Model C's water coefficient is -0.1439 MPa per kg/m^3. In words, our model predicts a 0.1439 MPa decrease in 28-day strength for a one kg/m^3 increase in water, holding cement constant. So, our model suggests a negative relationship between water and strength. While Regression Imputation has the highest R^2 value of 0.3303, the largest R^2 does not estalish the best future predictor. For one, the R^2 of Model A is incomparable to the R^2 Model B and C because they are fit on different rows; this is even more of an issue given that the remaining rows in Model A are not representative of the original data (we disproportionately removed high coarse aggregates). In this way,it can be argued that Model A is modeling the behavior of a completely different sub-population. Forms of imputation can also distort R^2 values based on the proposed patterns they create in the data, and there is practically no way to verify whether these patterns are close to true in a typical setting; for instance, its possible to perform an imputation that wrongly reshapes the structure of the data, but this wrong shape is better described by a line, resulting in higher R^2. Even if these issues were squared away, R^2 is in-sample, meaning that we are evaluating the model on points it was trained on. So, the metric is optimistically biased and does not accurately model the phenomenom of testing new points on a pre-fit model. In a sense, the metric rewards itself for capturing all forms of variation, regardless of if this variation is signal or noise.

Part B

When we treat deterministic imputations as known measurements, we misrepresent the missing-data in a fundamental way. Namely, the uncertainty estimates (such as confidence intervals) produced by the regression model are blind to imputation. That is, they see replaced values as observations who are as fixed as truly observed values. In reality, these imputed points are estimate, and the uncertainty of this estimate is entirely unaccounted for. To account for this, we employ impute-analyze-pool. First, we produce m plausible complete datasets (each gap is filled with the prediction plus a random draw of noise), and, for each one, we perform the same regression; this produces m water coefficients and m standard errors. Next, we pool their results. We begin by taking the average of the coefficient estimates, which will represent the estimated coefficient. Then, we average the m squared standard errors, which represents the ordinary uncertainty associated with complete data. From here, we compute the variance of the m regression coefficients, which represents the uncertainty brought on by the imputation step. Finally, we sum these variances (applying a corrective factor that is beyond the scope of this explanation) and take their square root, which represents the pooled standard error. Our confidence interval is then computed by using the estimated coefficient and the computed standard error. The key is that in this scheme, we now account for typical uncertainty AND the uncertainty brought upon by imputing the missing values.

Deterministic imputations make missing values an exact function of observed values. Even if this underlying relationship were true, we would still see small perturbations from this pattern. As a result, our current representation understates the variation of these points around the proposed relationship, making their fit look more perfect than it ever would. Adding noise aims to better represent the underlying data, but it is still a single imputation treated as known, which means we are not accounting for the uncertainty brought upon by the imputation process itself; this is why we need to perform repetitions in impute-analyze-pool.

Part C

Before any imputation or preprocessing, we should randomly partition our data into k folds. Then, hold out one of these folds. For the remaining folds, construct an imputation model that predicts water from the remaining predictors (excluding the outcome strength, because new mixtures won't have a strength to impute from). The important point here is that the imputation model is fit ONLY on the training data, meaning it DOES NOT include any held-out data (such as strength). Otherwise, our test data would be influencing the imputation model, which in turn will influence the predictive model we fit. So, when we would go to calculate test RMSE, our model would have already "peeked" at the test data via the imputation leakage; this leads to an overly optimistic evaluation. Next, we fit our prediction model using the imputed values along with our complete cases; again, this is only using the training data. Finally, we test this model (comprised of imputation and prediction) on our held out data by computing RMSE; if test observations are missing water, then apply the same, already fitted imputation model before applying the prediction model. Repeat this process for each fold, and average the RMSE across these folds. Once we complete our cross validation, we can choose to fit our imputation model and prediction model using all of the training data.

Note that we do not use random perturbations in this workflow. Random peturbations are used to restore the variation of our distribution so that inference procedures are more honest. When we are predicting, the best guess (in expectation) for water is always the conditional mean that our imputation model produces, not the conditional mean and a random perturbation. Adding a random perturbation will generally result in more variable predictions.